

Spoof-Detection — Kujur et al. (2024) INS monitor with Section-5 recovery

MATLAB implementation of the CPI + solution-separation spoofing monitor from Kujur, Khanafseh & Pervan, *Optimal INS Monitor for GNSS Spoofing Error Detection*, NAVIGATION 71(1) 2024 (navi.629), with the post-detection recovery FSM (NOMINAL -> COAST -> PROBATION -> handback), packaged for a two-rate host: a 100 Hz navigation loop and a 2 Hz GNSS/monitor task.

Runtime path (what the target simulation needs)

Host contract: docs/HOST_2HZ_WIRING.m. The gate works in update-increment space ($x+ - x_bar$), so it is independent of the host's feedback/gain bookkeeping of KF.states.

file	role
SPF_gate.m	persistent-state 2 Hz gate; wraps SPF_protectedNav
SPF_protectedNav.m	FSM in increment form; host owns its filters, gate returns corrections
SPF_monitorPool.m	overlapping-window bank (N slots)
SPF_cpiMonitor.m, SPF_ssMonitor.m	per-window CPI (Eq. 33/35) and SS (Eq. 49-52) tests
SPF_insCoast.m, SPF_revalidation.m	INS-only propagation (E28/E31), chi-square re-validation
matrixInv.m	STAND-IN for the simulation's SVD inverse (<code>[inv, invalid] = matrixInv(A)</code>), used by the CPI and re-validation on the zero-padded covariances; replace with the simulation's own file
STRUCT_SPF.m	all struct constructors + window open/close
CST_spfParam.m	solved monitor constants (single source of truth at runtime)
CST_spfMode.m	mode enumeration
CST_gnssHybrid.m	stand-in (<code>NO_STATES = 60</code>); the simulation has its own

Harness / offline design (not deployed)

run_recovery.m (flat 2 Hz demo + offline design), run_scheduler_test.m + scheduler_sim.m (100 Hz + 2 Hz split-rate architecture test), generate_test_data.m, build_Phi_Q.m, kalman_update_step.m, kujur_params.m, solve_N_min.m, compute_PMD_eq38.m, recovery_nav_sim.m (a host following docs/HOST_2HZ_WIRING.m), ins_100hz_template.m (what the host's 100 Hz function must add).

Tests

Run runAllTests from the repo root before integrating (MATLAB; for Octave see tools/octave_shim/README.md). Do not add tools/octave_shim to a MATLAB path. It runs the four scripts below in order and prints one PASS/FAIL line each.

- run_scheduler_test.m — 100 Hz + 2 Hz architecture (T1-T6)
- tests/test_monitors.m — engine unit + Monte-Carlo checks (replaces the dual_monitor-era test_all.m, which cannot drive SPF_cpiMonitor since the window length / thresholds moved into CST_spfParam)
- tests/test_variable_numMeas.m — varying satellite count, padded vs exact
- tests/test_feedback_split.m — host setKF feedback split (GAIN), drain in COAST
- tests/test_error_handlers.m — unusable covariances via matrixInv, numMeas clamp, NaN input, anchor stamp
- tests/test_matrixInv.m — contract of the host's SVD inverse: zero padding, accuracy on code+carrier S, non-finite flag, rank deficiency; run it with the simulation's own matrixInv on the path
- tests/test_shadow_mode.m — guide Step 5: shadow mode on clean data (zero alarms), then a scripted attack with authority (latch, vetoes, one commit)
- run_recovery.m — end-to-end demo with offline design and plots

Host decisions

`docs/HOST_DECISIONS.md` tabulates, per mode and event, what `setKF` sets (`stateFB`, `KF.states`, `KF.covariance`), how the trial is handled, what each mode compares against, and the gate outputs. `docs/HOST_2HZ_WIRING.m` is the code-level contract.

Logging and diagnostics

`tools/SPF_logInit.m`, `tools/SPF_logAppend.m` and `tools/SPF_plotLog.m` (MATLAB desktop, not code generation) record the gate telemetry once per 2 Hz epoch and draw the standard figures: mode timeline with events, SS/CPI margins per axis (alarm at 1), protection level, host NIS, re-validation statistic against its threshold, dwell, coast time, faults. The header of `SPF_plotLog.m` says how to read them for a jamming or false-latch investigation. The mode figure shades the warm-up: after a gate initialisation the monitors stay disarmed (`info.armed = false`, no latch possible) until the filter has had `ARM_EP0CHS` consecutive epochs with enough rows and a protection level below `ARM_PL_MAX` (`CST_spfParam`).

Printable copies

`docs/pdf/` holds PDF renderings of this README, `docs/HOST_DECISIONS.md`, `docs/REVIEW_FINDINGS.md` and `docs/CONCEPTS.md` (separation, coast, `P_C` / `P_KF`, anchor and warm-up explained in question-and-answer form), plus `ALL_DOCS.pdf` with all of them in one file. Regenerate after editing the Markdown with python3 `tools/docsToPdf.py` (needs `pip install markdown` and a Chromium binary).

Code generation

The runtime path (`SPF_gate` and everything it calls) is written for MATLAB Coder: fixed-size padded buffers, one struct layout per type from `STRUCT_SPF`, `uint8` enumeration modes, no cells, `varargin`, dynamic fields, function handles or `try/catch`. `tools/codegenSpoofMonitor.m` (MATLAB only) runs `coder.screener` on the call tree and builds a C library with the entry-point types taken from the `STRUCT_SPF` zero constructors. The gate must only run in navigation mode (see the alignment section of `docs/HOST_2HZ_WIRING.m`).

Regression

`reference/` holds the console output of both scripts from the author's MATLAB run; the current engine reproduces it exactly on MATLAB. `tools/octave_shim/` lets everything run under GNU Octave (event epochs differ by the RNG stream).